

손찬양

Java · Spring Boot 백엔드 개발자

5년 경력 · API 개발·운영 · 데이터 정합성

이메일: cyson21@kakao.com GitHub: github.com/cyson21 포트폴리오: cyson21.github.io

요약

- 기업용 플랫폼의 API와 이벤트 모듈을 Java·Spring Boot로 개발·운영해 온 5년 경력 백엔드 개발자입니다.
- 운영 오류를 재현해 상태 변경·시간대 처리 규칙과 회귀 테스트를 보강하고, 구형 AWS SDK 전환과 DB 통합 테스트 표준화를 수행했습니다.

경력

이엠캐스트(주)

2024.03 ~ 재직 중

백엔드 개발자 · 주임

기업용 플랫폼의 API와 이벤트 기능을 개발·운영하며 요구사항 분석, API와 데이터 저장 구조 설계·구현, 회귀 테스트를 담당했습니다.

- 상태 변경 충돌 방지:** 화면을 연 뒤 저장하기 전에 처리 상태가 바뀌면 오래된 기준으로 저장되어 진행 상태와 완료 여부가 어긋나는 경로를 추적했습니다. 저장 직전에 최신 상태를 다시 확인하고 유효하지 않은 변경을 차단하도록 수정했습니다.
- 시간대 변환 오류:** UTC 기준 날짜를 KST 일정으로 변환하는 과정에서 알림이 하루 늦게 실행되거나 누락되는 오류를 재현했습니다. 날짜 경계의 변환·발송 규칙을 수정하고 회귀 테스트로 경계 조건을 확인했습니다.
- 조회·삭제 조건 일치:** 같은 사용자 ID에 서로 다른 유형의 데이터가 연결될 때 일부 행이 누락되고 페이지 합계가 달라지는 원인을 확인했습니다. ID와 데이터 유형을 함께 비교하도록 QueryDSL 조회·삭제 조건을 통일하고 통합 테스트로 검증했습니다.
- AWS SDK v2 전환:** S3 연동 코드에 혼재한 구형 AWS SDK를 v2로 전환하고 업로드·인증 방식을 통일했습니다. 구형 의존성 제거, 모듈 테스트와 빌드 산출물 검사로 전환 범위를 확인했습니다.
- DB 테스트 표준화:** JPA 테스트마다 반복되던 수동 설정과 컨테이너 구성을 Testcontainers MySQL 기반 공통 구성으로 표준화했습니다. 공통 설정만으로 QueryDSL 조회와 시간대 변환을 실제 MySQL에서 반복 검증할 수 있게 했습니다.

공공·실시간 데이터의 수집·가공과 REST API 제공, Docker 배포·운행을 담당하고 시계열 예측 결과를 서비스 지표와 기능에 연동했습니다.

- 서울 실시간 도시데이터 Open API를 수집·가공하고 Spring Boot와 FastAPI로 조회 API를 제공했습니다.
- 서비스별 조회·저장 요구사항에 맞춰 MySQL·MongoDB 스키마와 Spring Boot·Django·FastAPI REST API를 설계·구현했습니다.
- 시계열 예측 결과를 서비스 지표와 기능에 연동하고, 관련 서비스를 Docker 컨테이너로 배포·운영했습니다.

기술

언어: Java, Python, SQL

프레임워크·ORM: Spring Boot, Spring Data JPA, QueryDSL, Django, FastAPI

데이터: MySQL, PostgreSQL, MongoDB

인프라·메시징: AWS S3, Docker, Kafka, Redis, RabbitMQ

테스트·도구: JUnit, Testcontainers, REST Docs, Git

대표 프로젝트

StockRush

분산 정합성 · Kafka · Saga · Transactional Outbox

문제: 주문·결제·재고가 분리된 흐름에서 중복 이벤트, 늦은 결제 승인, 발행 재시도로 주문 상태와 Outbox가 어긋날 수 있었습니다.

보호 규칙: Saga 보상 흐름과 Transactional Outbox를 구성하고, 소비자 먹등 처리와 취소·완료 상태의 후속 변경 차단을 보호 규칙으로 두었습니다.

검증: 중복 이벤트를 2회 받아도 Outbox 1건만 유지되고 취소 후 늦은 승인이 무시됨을 확인했습니다. 발행 실패를 5회 재현했을 때 Outbox 이벤트가 FAILED 상태로 전환됨을 통합 테스트로 검증했습니다.

github.com/cyson21/stockrush

Member Event Consistency

동시성·정합성 · PostgreSQL · Redis · RabbitMQ

문제: 동시 요청에서 최초 보상이 중복 지급되거나 쿠폰이 정원을 초과해 발급되고 포인트 잔액이 음수가 될 수 있었습니다.

보호 규칙: PostgreSQL 제약과 조건부 갱신을 최종 보호 규칙으로 두고, Redis 잠금과 RabbitMQ 단일 소비자 처리의 역할을 분리했습니다.

검증: PostgreSQL Testcontainers에서 최초 보상 동시 요청 8건 중 1건만 반영됐고, 60포인트 차감 2건은 1건 성공·1건 거절·잔액 40으로 수렴했습니다. RabbitMQ Testcontainers에서는 정원 3건에 발급 3건·거절 5건을 검증했습니다.

github.com/cyson21/member-event-consistency

학력

서울과학기술대학교 · 학사 / 영어영문학 · 2018 졸업